

Guía de Trabajo — Mini Agar.io en Pygame

Nivel:— Área 1 | **Modalidad:** Individual | **Duración:** 180 minutos

Objetivo

Desarrollar un prototipo tipo Agar.io en Pygame con un mundo más grande que la pantalla, cámara centrada en el jugador, crecimiento por masa, bots con IA básica (huir/perseguir), y una mecánica de dash que sacrifica masa. El propósito es comprender física simple, colisiones aproximadas, escalado de dificultad y diseño de mecánicas emergentes.

Mecánicas clave del prototipo

- Movimiento con mouse (dirección hacia el cursor).
- Comer comida y blobs más pequeños para crecer ($\text{radio} \sim \sqrt{\text{masa}}$).
- Velocidad se reduce conforme aumentas de tamaño.
- Dash [SPACE]: gana impulso hacia el cursor y pierde 5% de masa.
- IA simple de bots: huyen de más grandes, persiguen presas cercanas.
- Win: sin bots vivos. Game Over: si un bot grande te engulle.

Entregables (GitHub + Sites)

1) Repositorio en GitHub con estructura sugerida:

```
/src/main.py  
/docs/Manual_Tecnico.md  
/docs/Manual_Usuario.md  
README.md  
.gitignore (Python)
```

2) Manual Técnico (arquitectura del juego, cámara, colisiones, IA, parámetros y balance).

3) Manual de Usuario (instalación, ejecución, controles, solución de problemas).

4) Google Sites (individual): objetivo (2–4 líneas), bitácora técnica, 1–2 videos (≤ 10 s), respuestas de investigación (mínimo 12) con 3–5 fuentes, enlace al repo y al release.

Actividades en clase (sugerencia de tiempos)

- 1) Exploración (15 min): ejecutar, revisar controles y HUD.
- 2) Mundo y cámara (20 min): explicar WORLD_W/H y seguimiento con lerp.
- 3) Crecimiento y balance (25 min): $\text{radio} \sim \sqrt{\text{masa}}$, velocidad dependiente del tamaño.
- 4) IA y colisiones (35 min): criterio de presa/amenaza, colisión aproximada por distancia.
- 5) Dash y riesgos (20 min): perder masa para ganar movilidad; cuándo conviene.
- 6) Pulido y documentación (20 min): comentarios, parámetros, README; subir a GitHub.

Retos (elige 2 o más)

- Split [Q]: dividir al jugador en 2 células y fusionar tras 6 s.
- Expulsar masa [E]: genera comida delante del jugador con cooldown.

- Zoom dinámico: escalar la vista según tamaño del jugador.
- Skins y bordes: estilos de trazo/gradientes con primitivas.
- 2º jugador local (WASD) para pruebas PvP.
- QuadTree para acelerar detección (investigación + pseudocódigo).

Preguntas de investigación (responder mínimo 12)

- 1) Diferencias entre detección de colisiones por radio vs AABB (ventajas/costos).
- 2) ¿Por qué el radio suele escalar con $\sqrt{\text{masa}}$ en Agar.io? Justifica con geometría.
- 3) Modelos simples de desaceleración según tamaño: ¿lineal, logarítmico, por potencia?
- 4) ¿Qué es lerp y cómo se usa en la cámara? Ventajas y posibles artefactos.
- 5) Diseño de IA reactiva mínima: reglas de huir/perseguir y umbrales.
- 6) ¿Qué técnicas existen para reducir costo de colisiones en mundos grandes (grids, QuadTrees)?
- 7) Impacto del 'dash' en el metajuego: riesgos, coste-beneficio y anti-abuso (cooldowns).
- 8) Balance de spawn de comida: densidad, reposición y límites.
- 9) HUD claro: qué métricas mostrar y por qué (masa, velocidad, zoom).
- 10) Buenas prácticas de commits, issues y releases en GitHub para proyectos de juego.
- 11) Diseño de partículas/efectos para feedback sin saturar la escena.
- 12) Accesibilidad y legibilidad en mapas grandes: contraste, guías visuales, minimapas.
- 13) ¿Qué considerarías para multijugador real (sincronización, lag, autoridad del servidor)?
- 14) Seguridad en dependencias y gestión de versiones (requisitos.txt).

Rúbrica (20 puntos)

Criterio	4	3	2	1
Mecánicas (movimiento, comer, dash)	Funciona y está balanceado	Funciona con detalles menores	Inconsistente	No funcional
Mundo y cámara	Correctos y suaves	Correctos con leves artefactos	Problemas notables	Deficientes
IA y colisiones	Reglas coherentes y eficientes	Correctas con mejoras posibles	Limitadas/inestables	Ausentes
Código y documentación	Ordenado + Manuales completos	Adecuado	Incompleto	Deficiente
Investigación y evidencias	≥12 respuestas + 3-5 fuentes + capturas	Parcial	Insuficiente	Ausente